Lab Guide

Lecture 07 — Build a Zigbee Network and Demonstrate It

Communication Technologies for the Internet of Things

Academic year 2026–2027

Catarina Silva
ESTGA, University of Aveiro

Overview

This is the first Zigbee lab guide. The theory part introduced IEEE 802.15.4, the coordinator, router and *end device* roles, the topologies, addressing and security. In this guide everything is brought together in a system that works end to end and, above all, that system is used to *observe and measure* the behaviour of a Zigbee network.

The system is a monitoring and control station for a room. A CC2531 USB dongle, plugged into a Raspberry Pi, acts as the network coordinator. The Zigbee2MQTT program translates what happens in the Zigbee network into MQTT messages, published on a Mosquitto broker, and Node-RED displays the state of the devices and sends them commands. What is assessed at the end is not the dashboard itself, but what the group can demonstrate and explain about the communication technology behind it.

The guide reuses last year's material (installation of Zigbee2MQTT and Mosquitto, device pairing, network configuration, web interface and traffic capture with the CC2531 sniffer) and adds the format of Lecture 06: a system to build, a catalogue of results to demonstrate and a theoretical background.

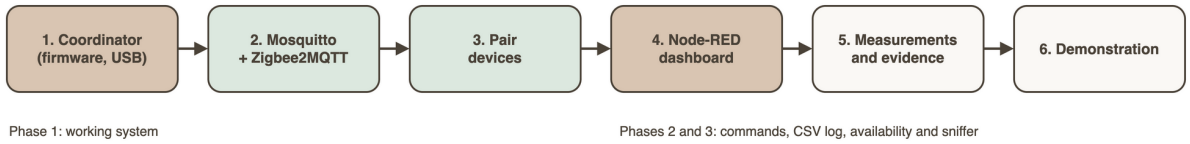


Figure 1: *Stages of the guide*

Material

- Raspberry Pi with a prepared card (Debian or Ubuntu Linux), SSH enabled and connected to the laboratory local network.
- CC2531 dongle with the coordinator firmware `CC2531ZNP-Prod.hex`, plugged into the Raspberry Pi.
- At least one Zigbee device. A smart plug (for example the SONOFF S26R2ZB used last year) acts as a router and accepts commands; a battery-powered sensor, if available,

acts as an *end device*.

- A second CC2531 dongle with the `sniffer_fw_cc2531.hex` firmware, for traffic capture (optional).
- Personal computer, for SSH access, to view the dashboard and for Wireshark.
- Folder `codigo/`, with `configuration.yaml`, `medidor_zigbee.py`, `modelo_topologia.js` and the `firmware/` subfolder.

Each group needs one coordinator and one Raspberry Pi. If one of them is missing, two groups share the material and take turns, because each network has a single coordinator. While one group measures, the other prepares the dashboard, reads the code or works on the model of category H, which needs no equipment.

Learning Objectives

- Integrate a Zigbee network, an MQTT gateway and a visualisation into a single working system;
- Measure quantities that characterise a Zigbee network (*link quality*, command latency, report rate, failure detection time);
- Relate what is observed to the protocol (channel, PAN ID, roles, topology, *join*, security keys);
- Compare the resilience of tree and mesh topologies;
- Support each claim with evidence: a number, a chart or a capture.

1. System to Build

Connections

The coordinator dongle plugs into the Raspberry Pi over USB and shows up in Linux as a serial device. The second dongle, with the sniffer firmware, plugs into the personal computer (or the Raspberry Pi) and only listens: it does not belong to the network.

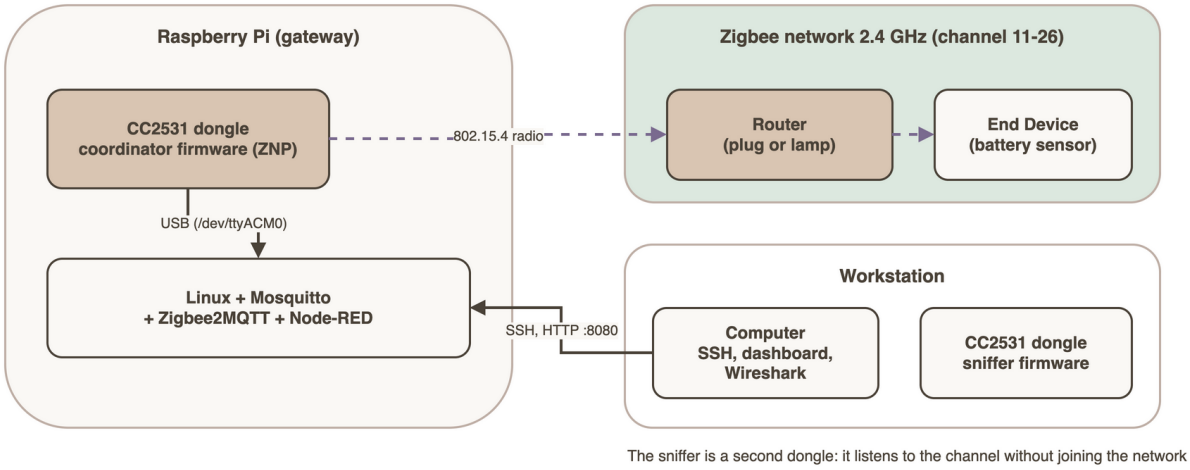


Figure 2: System connections

Architecture

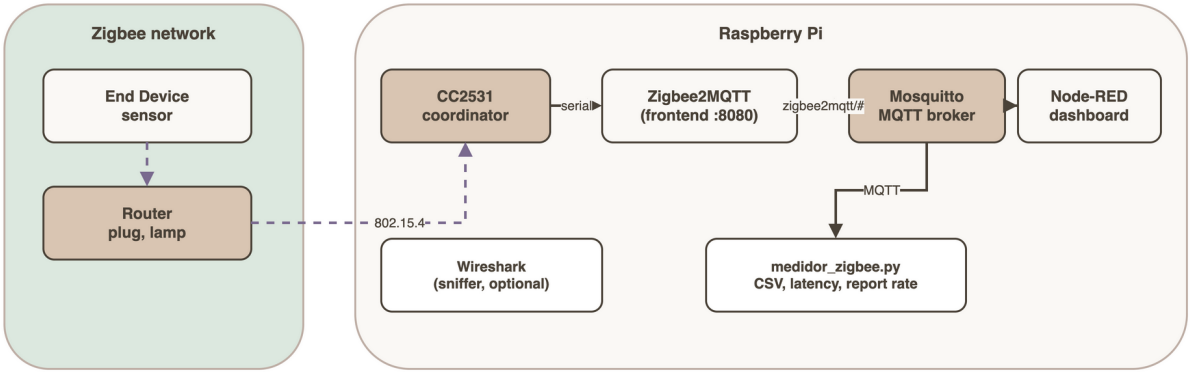


Figure 3: System architecture and what runs on each element

Accessing the Raspberry Pi

In a real installation the gateway sits next to the devices, without a monitor or keyboard. The work is therefore done over SSH from the personal computer.

```
ssh pi@<RASPBERRY_ADDRESS>
cat /etc/os-release
lsusb | grep -i texas
ls -l /dev/serial/by-id/
```

The `lsusb` command should list the CC2531, and the `/dev/serial/by-id/` folder the stable path of the serial port, which is what Zigbee2MQTT needs. If the port cannot be opened, add the user to the `dialout` group (`sudo usermod -aG dialout $USER`) and log in again.

Coordinator firmware

The CC2531 is a microcontroller with an IEEE 802.15.4 radio that only behaves as a Zigbee coordinator after it receives the right firmware. The laboratory dongle already carries `CC2531ZNP-Prod.hex`, the Z-Stack image in ZNP mode (*Zigbee Network Processor*), in which the

dongle handles the radio and the stack and the Raspberry Pi controls it over the serial port. If it has to be flashed again, a programmer (CC Debugger) and the procedure from the official Zigbee2MQTT guide for the CC2531 are needed. The same applies to the sniffer dongle, which receives `sniffer_fw_cc2531.hex`. Both files are in `codigo/firmware/`.

Mosquitto and Node.js

The guide follows the Zigbee2MQTT instructions for Linux installation (Ubuntu option). For Node.js the NodeSource repository for version 20 is used, as last year, instead of what the guide indicates; always confirm the supported version in the official documentation.

```
sudo apt update
sudo apt install -y mosquitto mosquitto-clients git make g++ gcc
   libsystemd-dev
sudo curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs
sudo corepack enable
```

Installing Zigbee2MQTT

```
sudo mkdir /opt/zigbee2mqtt && sudo chown -R $USER: /opt/zigbee2mqtt
git clone --depth 1 https://github.com/Koenkk/zigbee2mqtt.git /opt/
   zigbee2mqtt
cd /opt/zigbee2mqtt
pnpm install --frozen-lockfile
```

In older Zigbee2MQTT versions, such as last year's 1.33, dependencies were installed with `npm ci` and started with `npm start`. The official guide also explains how to run Zigbee2MQTT as a service with `systemctl`, which is advisable for a permanent installation.

Network configuration

The network is defined in `data/configuration.yaml`. Copy `codigo/configuration.yaml` and adjust the serial port, the channel and the PAN ID. The channel and the PAN ID identify the network; the channel should stay away from Wi-Fi channels 1, 6 and 11, which overlap the Zigbee band; the 128-bit network key is generated on first start. The web interface (*frontend*) is available at `http://<RASPBERRY_ADDRESS>:8080` and shows the devices, their capabilities, the log and the network map.

Start-up and pairing

```
cd /opt/zigbee2mqtt
pnpm start
```

On start-up the log shows the version, the coordinator firmware, the number of devices already joined and the connection to the broker. A device only joins the network when joining is permitted and the device itself is in pairing mode (for a plug, typically a button held for a few seconds).

Joining is opened in the web interface (*Permit join* button) or over MQTT, and should be closed again afterwards, because during that time any device can join the network.

```
mosquitto_pub -t zigbee2mqtt/bridge/request/permit_join -m '{"time":
   60}'
mosquitto_sub -t 'zigbee2mqtt/#' -v
```

After joining, the device is queried in an *interview*, in which Zigbee2MQTT reads the endpoints and clusters it offers, and it is identified from a database of supported devices. Last year's log, with a SONOFF plug, shows the sequence:

```
Device '0x00124b0024c87aaf' joined
Starting interview of '0x00124b0024c87aaf'
Successfully interviewed '0x00124b0024c87aaf', device has successfully
   been paired
Device '0x00124b0024c87aaf' is supported, identified as: SONOFF Zigbee
   smart plug (S26R2ZB)
```

An unsupported device also joins the network, but only becomes usable after a definition is added for it, as described in the documentation on supporting new devices. The initial name is the 64-bit IEEE address; give each device a readable name in the web interface, for example `plug1`.

Node-RED dashboard

Each device publishes its state on `zigbee2mqtt/<name>` and receives commands on `zigbee2mqtt/<name>/set`.

1. Drag an `mqtt in` node, connected to the broker `localhost:1883`, subscribed to `zigbee2mqtt/plug1`, with a JSON object output.

2. Drag a `function` node with two outputs and paste the following content:

```
var d = msg.payload;
return [ { payload: d.linkquality }, { payload: d.state } ];
```

3. Connect the first output to a `ui-chart` named *Link quality* and the second to a `ui-text` named *State*.

4. To send commands, drag a `ui-switch` whose output goes to a `function` returning `{payload: {state: msg.payload ? "ON" : "OFF"}}`, connected to an `mqtt out` with the topic `zigbee2mqtt/plug1/set`.

5. Press `Deploy` and open the dashboard.

Phases

The work is done in three phases, each building on the previous one. For the demonstration it is enough to have the first phase working.

1. In the first phase Zigbee2MQTT is running, at least one device is paired and the Node-RED dashboard shows its state and *link quality*.

2. In the second phase the meter is used to record messages, latencies and report rates in a CSV file, availability monitoring is enabled, and the dashboard gains device control and an on/off indicator.

3. In the third phase the network traffic is captured with the sniffer and analysed in Wireshark, and the real network map is compared with the model of category H.

The meter

The meter is supplied ready to use and talks to Zigbee2MQTT only over MQTT. The task is to run it, read it and adapt it, for example to add a field to the file or to measure another device. The mode names are in English (`devices`, `permit`, `listen`, `lqi`, `rate`, `latency`, `map`); the Portuguese names also work.

```
uv run codigo/medidor_zigbee.py devices
uv run codigo/medidor_zigbee.py listen 60 readings.csv
uv run codigo/medidor_zigbee.py lqi plug1 30
uv run codigo/medidor_zigbee.py latency plug1 50
uv run codigo/medidor_zigbee.py rate sensor1 600
uv run codigo/medidor_zigbee.py map
```

Zigbee2MQTT options

The request topics, such as `zigbee2mqtt/bridge/request/permit_join` or `networkmap`, and the format of their parameters change slightly between Zigbee2MQTT versions. If a request gets no answer, check the documentation of the installed version. Availability monitoring (`availability: true`) makes Zigbee2MQTT publish the `online` or `offline` state of each device on `zigbee2mqtt/<name>/availability`.

Traffic capture

The sniffer listens on one channel and sends the IEEE 802.15.4 frames to Wireshark, which decodes them as Zigbee. Flash `sniffer_fw_cc2531.hex` on the second dongle and follow the procedure in the Zigbee2MQTT guide (section *Sniff Zigbee traffic*) or the community guide listed in the references. The sniffer channel must be the network channel. To decrypt the frames, Wireshark needs the network key, which is in `configuration.yaml`, and the default link key of Zigbee 3.0.

2. Results to Demonstrate

With the system working, each group chooses the results it wants to demonstrate. Each row of the following tables describes a possible result and the evidence that supports it. The level indicates the relative effort: 1 is a direct observation, 2 requires a repeated measurement and 3 requires a measurement, a capture or a change to the system.

A. Network and discovery

Id	Result to demonstrate	Evidence	Level
A1	A new device joins the network and is identified (*interview*), with the vendor, the model and the capabilities it exposes.	Zigbee2MQTT log and web interface screen.	1
A2	Each device has a fixed 64-bit IEEE address and a 16-bit network address assigned on joining.	Table with both addresses of each node, obtained from the meter, and an explanation of the difference.	1
A3	The role of each device (coordinator, router, *end device*) and how it is powered.	Output of `devices` and a justification based on how each is powered.	1
A4	Channel and PAN ID of the network, and how the channel relates to neighbouring Wi-Fi networks.	Network channel, Wi-Fi channels detected in the room and a discussion of the overlap.	2
A5	Time between the start of pairing and the end of the *interview*.	At least three repetitions, with the log times, and the mean.	2
A6	With joining closed, a device in pairing mode does not join.	Log of the refused attempt and of the attempt with joining open.	2

B. Link and quality

Id	Result to demonstrate	Evidence	Level
B1	The *link quality* (LQI) decreases with distance, but with large variability.	LQI chart at no fewer than four distances, 30 samples each, with mean and standard deviation.	2
B2	Effect of obstacles (body, wall, metal drawer) on the LQI at the same distance.	Comparison table and a conclusion on how useful the LQI is as a distance measure.	2
B3	The route has hops: a distant device connects through a router and not directly to the coordinator.	Network map with the links, their LQI and the number of hops.	3
B4	Adding a router halfway along a path improves the link of a distant node.	LQI of the distant node before and after switching on the router, with the network map.	3

C. Commands and reports

Id	Result to demonstrate	Evidence	Level
C1	Command latency: time between the request and the state confirmation.	Histogram or table of at least 50 samples, with median and maximum.	2
C2	Actual report rate of a sensor compared with the expected one (*reporting*).	Intervals between consecutive messages over at least 10 minutes.	2
C3	Command from the dashboard to the device: the plug switches and the confirmation returns to the dashboard.	Live demonstration and measured time.	1
C4	An *end device* only receives commands when it wakes up, unlike a router.	Response time of a router and of an *end device* to the same kind of request.	3

D. Data model and capacity

Id	Result to demonstrate	Evidence	Level
D1	Structure of a device: the endpoints, clusters and attributes it exposes.	Listing taken from the web interface and its match with what the dashboard uses.	1
D2	The same state seen at three levels: ZCL attribute, MQTT message and captured frame.	The three records side by side for one switching event.	3
D3	Header overhead: useful octets compared with the total octets of a data frame.	Calculation based on a captured frame.	3
D4	Message rate a device sustains without losses.	Messages per second at the limit, with a description of the method.	3

E. Robustness

Id	Result to demonstrate	Evidence	Level
E1	Time the system takes to mark a device *offline* when it loses power.	Measurement repeated at least three times and its relation to the availability parameter.	2
E2	Automatic recovery: the device rejoins the network by itself after power is restored, without new pairing.	Log with the time of the cut, the time of the return and the state on the dashboard.	2
E3	Restarting Zigbee2MQTT keeps the network and the devices.	Start-up log (*started (resumed)*) and devices still present.	1
E4	Effect of interference in the 2.4 GHz band (Wi-Fi, microwave oven, human body).	Losses or LQI with and without the interference source.	3

F. Protocol observation

Id	Result to demonstrate	Evidence	Level
F1	The sequence of joining a network: beacon request, beacon, association, device announcement and key transport.	Annotated sniffer capture, one step per line.	3
F2	The layers of a frame: PHY, MAC, NWK, APS and ZCL, with the fields of each.	A captured frame broken down in Wireshark.	3
F3	Without the network key the content is encrypted; with it, it is readable.	Capture before and after configuring the key, and an explanation of AES-CCM*.	3
F4	Channel and PAN ID seen in the capture match the configuration.	Frame and *beacon* fields compared with `configuration.yaml`.	2

G. Privacy and security

Id	Result to demonstrate	Evidence	Level
G1	With joining open, any device can join the network.	Joining of an unexpected device and a proposed mitigation (short windows).	2
G2	Whoever holds the network key from `configuration.yaml` can decrypt the traffic.	Decryption of a capture with that key and a discussion of key storage.	3
G3	The IEEE address of each device is visible in clear in the network frames.	Capture showing the address and a discussion of privacy.	2

H. Topology and resilience (model)

This category needs no equipment. It uses the Node-RED flow in `codigo/modelo_topologia.js`, which represents the network in the following figure and computes which nodes become unreachable when a router fails.

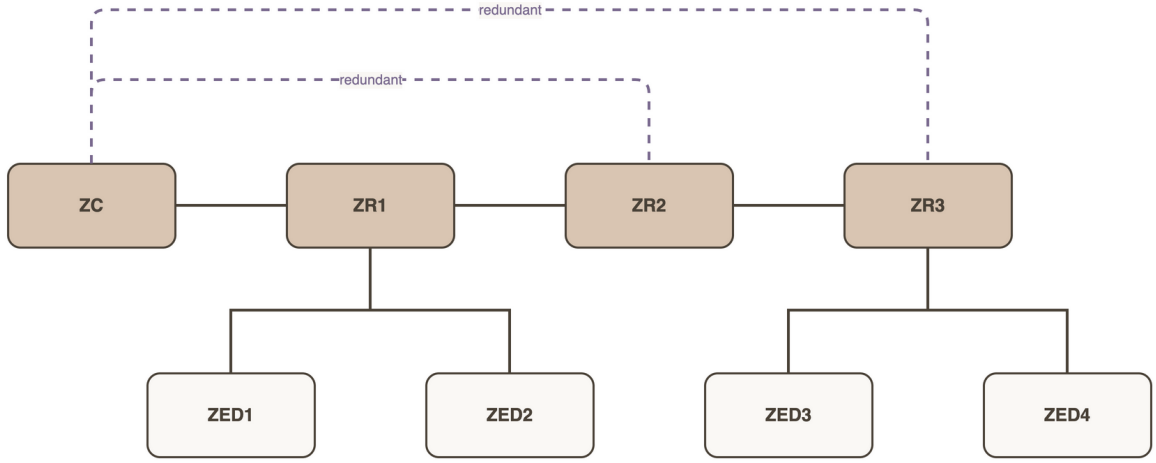
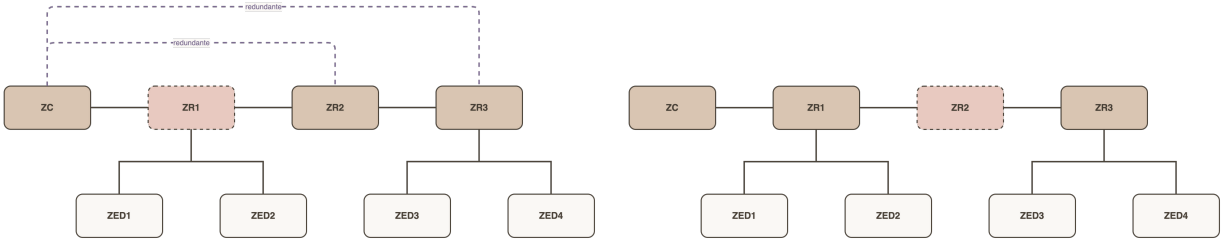


Figure 4: *Model network. Solid line: hierarchical link (parent and child); dashed: redundant link.*

Id	Result to demonstrate	Evidence	Level
H1	In a mesh, the failure of ZR2 isolates no node.	Model output for each failing router.	1
H2	In a tree (remove `ZC`–`ZR2` and `ZC`–`ZR3`), the failure of ZR2 isolates ZR3, ZED3 and ZED4.	Comparison table of mesh and tree.	2
H3	The failure of ZR1 isolates ZED1 and ZED2 in both topologies, which shows the limit of redundancy.	Model result and an explanation based on the single parent.	2
H4	The model reproduces the real map.	Real network map from the `map` mode compared with the model, listing the differences.	3



Mesh: failure of `ZR1` Tree: failure of `ZR2`

Figure 5: *Failing node dashed. Only the model result confirms the isolated nodes.*

3. Demonstration

At the end of the session each group shows the system working and presents the chosen results.

1. Show the system working, at least the first phase.
2. Choose at least three results, from at least two different categories, of which at least one is from categories A, C or F.
3. For each result prepare the evidence, which must be ready to show without new measurements.

Extra

What to do, step by step

1. Access the Raspberry Pi over SSH and confirm that the CC2531 dongle appears in `/dev/serial/by-id/`.
2. Install Mosquitto, Node.js and Zigbee2MQTT, and prepare `configuration.yaml` with the port, the channel and the PAN ID.

3. Start Zigbee2MQTT, open the web interface and confirm the coordinator firmware in the log.
4. Permit joining, put the device in pairing mode, wait for the *interview*, give it a name and close joining.
5. Build the Node-RED dashboard with the state, the LQI and the command, and confirm that the first phase works.
6. Run the meter, enable availability monitoring and add an on/off indicator to the dashboard.
7. Flash the sniffer, choose the network channel and capture a device joining in Wireshark.
8. Go through the tables of section 2 and choose the results to demonstrate.
9. Collect the measurements and the evidence for each chosen result.
10. Prepare the demonstration and present it to the instructor.

Without all the material

The guide needs the coordinator and the Raspberry Pi. Without Zigbee devices, or with only one, the available options are category A (with whatever exists), E3, F (if there is a sniffer, even when capturing traffic from other networks), G3 and category H, which uses only the Node-RED model. Without a sniffer, category F is replaced by B, C or E. With a single device there is no network map with hops, so B3 and B4 can only be done with at least two nodes besides the coordinator. The results in which the device is a plug (C1, C3) need an actuator; with a sensor use C2 and E1.

Other devices

Any device supported by Zigbee2MQTT will do, as long as the name and the fields read on the dashboard are adjusted: lamps, temperature and humidity sensors, contact or motion sensors, buttons and plugs with power metering. The results in the tables of section 2 concern the network and not the device type, so they still apply. If the measured quantity changes, the dashboard and the log file must change accordingly.

Theoretical background

This section describes systematically what the system built does and why it behaves as it does. Each result in section 2 corresponds to a protocol property described here.

Architecture and the role of the gateway

The system follows the usual three-element model of the Internet of Things. Devices connect to the physical world, sense and act, and have minimal processing, memory and energy resources. The gateway interconnects two networks of different nature, the Zigbee network and the local IP network. The consumer, represented by the dashboard and the log file, does not know Zigbee: it only receives MQTT messages already translated.

Zigbee2MQTT is the translation element. It receives the network frames from the coordinator, interprets them according to the *Zigbee Cluster Library* and publishes the result on MQTT topics; it does the reverse for commands. It also assigns readable names to devices, records their state and supervises availability.

Physical layer and medium access

Zigbee is based on IEEE 802.15.4, which in the 2.4 GHz band defines 16 channels, numbered 11 to 26, spaced 5 MHz apart, and a raw bit rate of 250 kbit/s with O-QPSK modulation and direct-sequence spread spectrum (DSSS). These channels coexist with Wi-Fi in the same band: a 20 MHz Wi-Fi channel covers several Zigbee channels, so the choice of the network channel (A4) looks for those that lie between or beyond Wi-Fi channels 1, 6 and 11. Medium access uses CSMA/CA, in which the node listens to the channel before transmitting and backs off if it is busy. Data frames are small, with a maximum of 127 octets at the physical layer, which limits the payload and gives weight to the headers (D3).

Roles and topology

The coordinator forms the network, chooses the channel and the PAN ID and acts as Trust Center, distributing the network key. Routers forward frames and allow new devices to join, and must always be active, so they are usually mains powered. *End devices* talk only to their parent, can sleep and therefore last years on a battery, but only receive when they wake up. In a mesh the routers form alternative paths; in a tree each node depends on a single parent. The failure of a router isolates the nodes that depend on it and have no other path, which is what category H shows.

Addressing and routing

Each device has a fixed 64-bit IEEE address and receives a 16-bit network address from its parent when it joins. Mesh routing uses the AODV algorithm, which discovers routes on demand and stores them in tables; in the tree topology routing follows the address hierarchy. The LQI that Zigbee2MQTT publishes, from 0 to 255, is an indicator of the quality of the last packet received, computed by the radio from the signal energy and correlation; it is relative and chip dependent, and therefore not a measure of distance (B1, B2).

Joining the network

A device that wants to join looks for networks by sending *beacon* requests on the various channels. The routers and the coordinator that permit joining answer with a *beacon*, the device requests association with one of them, which becomes its parent and assigns it the network address, and it announces itself to the network. The Trust Center then sends it the network key, encrypted with the link key. Only after this does Zigbee2MQTT run the *interview*. This is the sequence the sniffer shows in F1 and it only occurs while joining is permitted (A6, G1).

Data model

Each device exposes one or more *endpoints*, and each endpoint a set of *clusters* with attributes and commands. A plug has the On/Off cluster, with the state attribute and the *On*, *Off* and *Toggle* commands; a temperature sensor has the measurement cluster with the value in hundredths of a degree. The device can be queried (*read*) or configured to report by itself (*reporting*), with a minimum interval, a maximum and a minimum change. Spontaneous reports reduce traffic compared with periodic polling, and their interval determines the report rate observed in C2.

Security

All network frames are encrypted and authenticated with AES-128 in CCM* mode, using the network key, which is shared by all nodes, and a frame counter against replay. The network key is distributed on joining, encrypted with a link key. In Zigbee 3.0 the default link key is well known, which makes joining the most fragile moment, hence the recommendation to keep joining closed. Whoever holds the network key decrypts all the network traffic (G2); IEEE addresses, by contrast, travel in clear in the headers (G3).

Availability and recovery

Routers are always active and Zigbee2MQTT can test them at a short interval; *end devices* talk rarely, and therefore have a much longer silence limit before being declared *offline*. The failure detection time is thus very different between the two roles (E1). A device that is powered again looks for its parent and rejoins without new pairing, because the key and the PAN ID are stored (E2); the coordinator also stores the network state, and restarting Zigbee2MQTT resumes it (E3).

Measurement methodology

The results of the guide are measurements, and their value depends on the method. A measurement must state what was measured, under what conditions, with what instrument and with how many repetitions. For random quantities such as the LQI and latency, a single sample is insufficient: collect enough samples to estimate the mean and standard deviation, or the median and percentiles when the distribution is skewed, as it almost always is for latencies. The maximum value matters because in a real-time system it is the tail of the distribution that sets the worst case. Intervals are measured with a monotonic clock. The latency of a command includes routing in the Zigbee network, processing in the device, the return of the state report and the time in Zigbee2MQTT and in the broker. Change one variable at a time, keeping the position and the environment constant.

Discussion questions

1. What functions does the Zigbee2MQTT gateway perform that the device could not?
2. Why do *end devices* never appear as points of failure for other nodes?
3. The mesh topology has costs. What are they, compared with the star topology?

4. In a home automation installation with thirty devices on three floors, which topology would you choose and why?

5. Why is joining the network the most exposed moment, and what practice reduces the risk?

6. What happens to the network if the coordinator fails? And if it is replaced by another dongle?

References

- Zigbee2MQTT, Linux installation guide and configuration: `https://www.zigbee2mqtt.io/guide/getting-started/`
- Zigbee2MQTT, network configuration (channel, PAN ID, device joining): `https://www.zigbee2mqtt.io/guide/configuration/zigbee-network.html`
- Zigbee2MQTT, web interface: `https://www.zigbee2mqtt.io/guide/configuration/frontend.html`
- Zigbee2MQTT, sniffing Zigbee traffic: `https://www.zigbee2mqtt.io/advanced/zigbee/04_sniff_zigbee_traffic.html`; community guide: `https://community.oh-lalabs.com/t/guide-build-a-zigbee-cc2531-sniffer-how-to-use-it/469`
- Zigbee2MQTT, supporting new devices: `https://www.zigbee2mqtt.io/advanced/support-new-devices/01_support_new_devices.html`
- Node-RED documentation: `https://nodered.org/docs/user-guide/`
- IEEE Std 802.15.4-2020, *Low-Rate Wireless Networks*; Connectivity Standards Alliance, *ZigBee Specification* (05-3474).
- Gonçalves, P. Zigbee and Zigbee2MQTT course material, TCIoT, previous edition.
- Ramya, C. M. *et al.* (2015). A Survey of ZigBee Wireless Sensor Network Technology: Topology, Applications and Challenges. *IJCA* 130(9).